

科目 作業系統 學系 資訊工程 年級 2B

114 年 12 月 26 日 學號 01357101 姓名 黃翊宏 (Paper 1)

從此處開始寫起。試卷用紙務須節用。非經主試認可不得續用其他紙張作答。

記	分	教師簽名或蓋章

記分欄

1. Race Condition: 當多個 Process / Thread 同時存取並修改同一份資料時, 若沒有同步機制, 會導致實際執行結果仰賴程式的執行順序, 而非正確的預期結果。

Critical Section: Critical Section 是一段「受保護」的程式碼片段, 當一 Process / Thread 進入 Critical Section 時, 另一個 Process / Thread 就不能進入, 藉此來避免資料發生 Race Condition。

用流程式的方式寫

2. (c) turn 與 flag 的設定順序錯誤。若 Process i while (true) {
 設定完 turn = j 後, 馬上 Context Switch 到 Pj. flag[i] = true;
 (3%) 當 Pj 設定完 flag 後又發生 Context Switch, 這時就會導致同時有 2 個 Process 有進入 Critical Section 的 Permission. execution example, explain. while (flag[i] && turn == j);
 // critical section

→ (b) turn 應該要設給對方, 而非自己, 否則則 Pj 設定完 turn, Context Switch Pj, Pj 設定完 turn 又 Context Switch Pi, 會導致 flag[i] = false;
 3. 所有人皆無法進入 Critical Section. }

(a) sem = 1

(a) sync = 0

(b) 如果將 sem 的初值設為 0,

(b) 如果將 sync 的初值設為 1, Process 2

→ Process 就會一直持續卡在同一個地方, 不會進 waiting queue 中去等待。

→ 就會永遠等不到 Process 1 完成, 一直持續在等待。

4.

Spinlock 的實作方式是使用一個 while-loop 來持續「忙碌」的檢查進入 Critical Section 的條件是否被滿足。

advantage: 實作簡單, 適合 Critical Section 短, 預期很快就拿到鎖的場景。

Semaphores 則是讓不滿足進入 Critical Section 的 Process 直接放進 waiting queue 中去等待, 來避免發生如 spin lock 般忙碌等待的情況, 降低 CPU 資源的浪費。

advantages: 降低等待時 CPU 資源的佔用率, 適合 Critical Section 長的情況。

5.

- 1 mutual exclusion : 互相都需要對方所持有的那個資源。
- 2 hold and wait : 每個人都持有一個資源, 同時等待下一個資源。
- 3 no preemption : 不可以搶佔別人的資源。
- 4 circular wait : 允許 Process 隨意請求資源, 導致 A 等 B 的資源, B 等 C 的資源, 而 C 又回頭等 A 的資源。

method 1:

method 2:

→ 調整 lock 的順序, 來避免循環等待。

→ 不使用任何的 mutex lock, 來避免互斥 (mutual exclusion)

但這麼做可能會導致 Race Condition。

```
void do_work_one() {
    first_mutex.lock();
    second_mutex.lock();
    std::cout << "do_work_one" << std::endl;
    first_mutex.unlock();
    second_mutex.unlock();
}

void do_work_two() {
    first_mutex.lock();
    second_mutex.lock();
    std::cout << "do_work_two" << std::endl;
    first_mutex.unlock();
    second_mutex.unlock();
}
```

```
void do_work_one() {
    std::cout << "do_work_one" << std::endl;
}

void do_work_two() {
    std::cout << "do_work_two" << std::endl;
}

// ( 用 third_mutex.lock() / third_mutex.unlock()
// or try_lock()
```

6.

External Fragmentation: 總剩餘可用的記憶體

空間雖然足夠容納新的 Process, 但由於剩餘空間是不連續散落在記憶體中, 如同「空洞」一般, 而這些空洞沒有一處容納的下新的 Process, 引發記憶體不足。

7. page size 太大。

好處: page table 小, 管理容易, 降低額外的記憶體開銷。

壞處: internal fragmentation 影響更大, Page Fault 的代價更大。

Internal Fragmentation: 每個 Process 不可能都剛好 Page size 大小:

會是 frame size 的整數倍, 為了補齊這一誤差, 好處: 減緩 internal fragmentation.

會將 Process 填入空白區域來對齊到 frame size 的整數倍, 而填補空白造成的浪費就是 internal fragmentation 壞處: page table 變大, 管理難度增加。

8.

method 1: 透過階層式的方法來一一對應到下一層, 直到最後對應到實體記憶體。雖然層數的增加可能導致 Page Table 變多, 但每一層的對應都是 $O(1)$, 非常快速。

method 2: 透過 hash 的方式來管理 frame, 這樣就能有效降低 Page Table 的數量。但由於每一次對應之前都要先去表中搜尋對應的位址, 雖相比起階層式的管理減少了 Page Table 的數目, 但由於每一次搜尋查找都是 $O(n)$ 時間, 其效率相比起階層式還是慢上不少。

科目 作業系統 學系 資訊工程 年級 2B

114年12月26日 學號 01357101 姓名 (中文) 黃翊宏 (Paper 2)

從此處開始寫起。試卷用紙務須節用。非經主試認可不得續用其他紙張作答。

記分欄

分 教師簽名或蓋章

9. 1 2 3 4 5
 memory partitions: 180kb, 500kb, 260kb, 300kb, 620kb
 processes : 230kb, 390kb, 150kb, 450kb

First fit:

230kb → 500kb(2) [180kb, 270kb, 260kb, 300kb, 620kb]

390kb → 620kb(5) [180kb, 270kb, 260kb, 300kb, 230kb]

150kb → 180kb(1) [30kb, 270kb, 260kb, 300kb, 230kb]

450 → X external fragmentation

Best fit:

230kb → 260kb(3) [180kb, 500kb, 30kb, 300kb, 620kb]

390kb → 500kb(2) [180kb, 110kb, 30kb, 300kb, 620kb]

150kb → 180kb(1) [30kb, 110kb, 30kb, 300kb, 620kb]

450kb → 620kb(5) [30kb, 110kb, 30kb, 300kb, 170kb]

Worst fit:

230kb → 620kb(5) [180kb, 500kb, 260kb, 300kb, 390kb]

390kb → 500kb(2) [180kb, 110kb, 260kb, 300kb, 390kb]

150kb → 390kb(5) [180kb, 110kb, 260kb, 300kb, 240kb]

450kb → X external fragmentation.

10.

mem reference str. 6 7 2 0 2 3 2 5 1 0 3 2 3 0 2 0 1 6 0 7

* 3 frames

FIFO:

6 7 2 0 2 3 2 5 1 0 3 2 3 0 2 0 1 6 0 7

6 6 ~~7~~ 0 0 0 0 ~~1~~ 1 ~~2~~ 2 2 2 2 2 ~~0~~ 01 1 1 ~~2~~ 3 3 3 ~~5~~ 0 0 0 0 0 0 ~~1~~ 1 1 1M 2 2 2 2 ~~3~~ 5 5 ~~3~~ 3 3 3 3 3 3 ~~6~~ 6 6

✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓
 4 1 5 3

FIFO's Page Fault: 13次

記分欄

10.

Optimal:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
6	1	2	0	2	3	2	5	1	0	3	2	3	0	2	0	1	6	0	1
6	6	6 ₁₇	0	0 ₉	3	3 ₁₀	3	3 ₁₀	3	3 ₁₂	3	3	3	3	3	8 ₁₇	1 ₁₇	1	1
	1	1 ₈	1	1 ₈	1	1 ₈	1	1 ₁₆	1	1 ₁₆	2	2	2	2	2 ₂₀	2 ₂₀	6	6	6
M		2 ₄	2	2 ₆	2	2 ₁₁	5	5 ₁₀	0	0 ₁₃	0	0	0	0	0	0 ₁₈	0 ₁₈	0	0
	✓	✓	✓	✓		✓		✓	✓		✓					✓	✓	✓	x
	4				1	3								3					

Optimal Page Fault: 18次

LRU:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
6	1	2	0	2	3	2	5	1	0	3	2	3	0	2	0	1	6	0	1
6	6	6 ₁₇	0	0 ₃	0	0 ₃	5 ₇	5 ₇	5 ₇	3 ₁₀	3	3	3	3	3	8 ₁₇	1 ₁₆	1	1
	1	1 ₈	1	1 ₈	3	3 ₅	3 ₅	1 ₈	1 ₈	1 ₈	2	2	2	2	2 ₁₄	2 ₁₄	6	6	6
M		2 ₂	2	2 ₄	2	2 ₆	2 ₆	2 ₆	0 ₉	0 ₉	0	0	0	0	0	0 ₁₅	0 ₁₅	0	0
	✓	✓	✓	✓		✓		✓	✓	✓	✓	✓				✓	✓		
	4				1	5								2					

LRU Page Fault: 12次